

DocuMind AI

02 - Documento de Diseño

Sistema Inteligente de Gestión y Análisis Documental

Proyecto	DocuMind AI - Sistema Inteligente de Gestión y Análisis Documental
Autor	Miguel Andres Barrios Rodriguez
Asignatura	Desarrollo de Aplicaciones Empresariales - VI semestre
Versión	1.0
Fecha	Septiembre de 2026

Contenido

1. Propósito del diseño
2. Principios y alcance técnico
3. Arquitectura general de la solución
4. Arquitectura por capas
5. Diagrama de casos de uso
6. Diagrama de componentes
7. Diagrama de despliegue
8. Diagramas de secuencia
9. Modelo de datos
10. Diccionario de datos
11. Diseño de API y servicios
12. Diseño de interfaces y prototipos
13. Flujo de procesamiento documental
14. Diseño de la integración con IA
15. Diseño de búsqueda inteligente y RAG
16. Decisiones tecnológicas y justificación
17. Diseño básico de seguridad
18. Gestión de estados y errores
19. Consideraciones de escalabilidad
20. Trazabilidad diseño-requisitos
21. Conclusiones

1. Propósito del diseño

Este documento define la solución técnica de DocuMind AI a partir del alcance funcional del proyecto. El diseño cubre frontend, backend, datos, almacenamiento, procesamiento documental, inteligencia artificial, búsqueda semántica, seguridad, APIs y despliegue, de forma que la implementación pueda reproducirse y mantenerse alineada con los requisitos del análisis.

2. Principios y alcance técnico

- Arquitectura modular para separar interfaz, API, datos, archivos e IA.
- Soporte inicial obligatorio para PDF, DOCX y TXT.
- Procesamiento trazable mediante estados: CARGADO, EXTRAYENDO, PROCESANDO_IA, PROCESADO y ERROR.
- IA real desacoplada mediante un servicio interno, evitando acoplar el resto del sistema a un proveedor específico.
- RAG simplificado para responder preguntas con contexto recuperado de los documentos.
- MySQL como base transaccional y almacenamiento local de archivos para el prototipo académico.
- Seguridad básica de autenticación, credenciales, validación de archivos y control de acceso.

3. Arquitectura general de la solución

La solución se propone como una aplicación web cliente-servidor. El navegador consume una API REST desarrollada en Node.js/Express. MySQL almacena usuarios, repositorios, metadatos, resultados de IA, fragmentos y consultas. Los archivos se conservan inicialmente en una carpeta /uploads. El procesamiento de IA se encapsula en un servicio que puede consumir Gemini u otro proveedor equivalente.

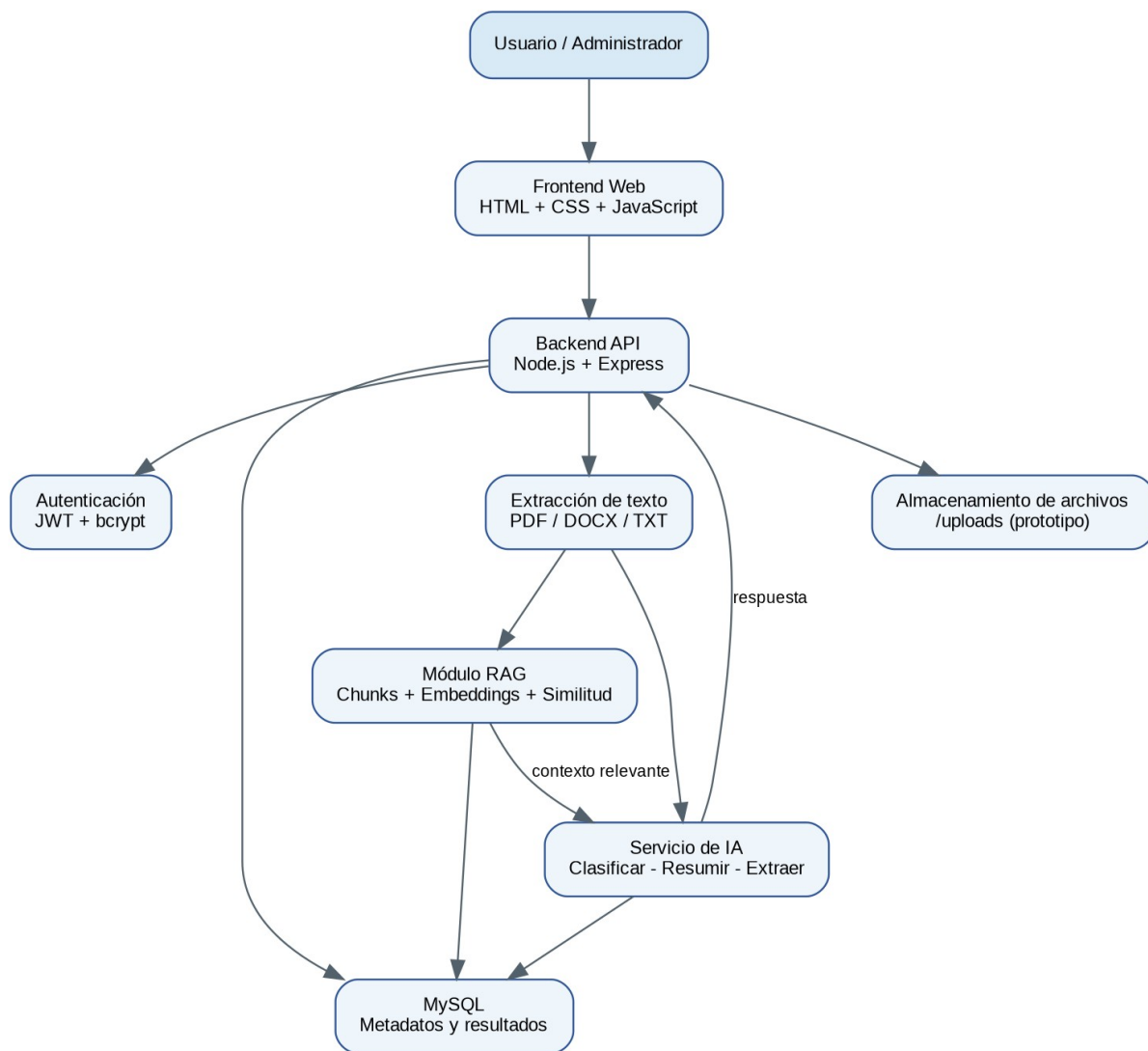


Figura 1. Arquitectura general propuesta de DocuMind AI.

4. Arquitectura por capas

Capa	Responsabilidad	Componentes propuestos
Presentación	Interacción con usuarios y visualización de resultados.	HTML, CSS, JavaScript, dashboard, formularios, chat documental.
API / Negocio	Validación, reglas de negocio, orquestación de procesos y seguridad.	Node.js, Express, controladores, servicios, JWT.
Datos	Persistencia estructurada y consultas.	MySQL, repositorios de datos, consultas parametrizadas.
Almacenamiento	Conservación de archivos originales.	Directorio /uploads en prototipo; almacenamiento cloud como evolución.
Procesamiento documental	Extracción y normalización del contenido.	pdf-parse, mammoth y fs.
IA	Clasificación, resumen, extracción y generación de respuestas.	AI Service + API de IA.

RAG / Búsqueda	Fragmentación, embeddings, similitud y recuperación de contexto.	RAG Service, chunks y embeddings almacenados.
----------------	--	---

5. Diagrama de casos de uso

El diseño contempla dos actores principales: Usuario y Administrador. Ambos se autentican; el usuario gestiona sus repositorios y documentos, mientras que el administrador incorpora tareas de administración, monitoreo y soporte.

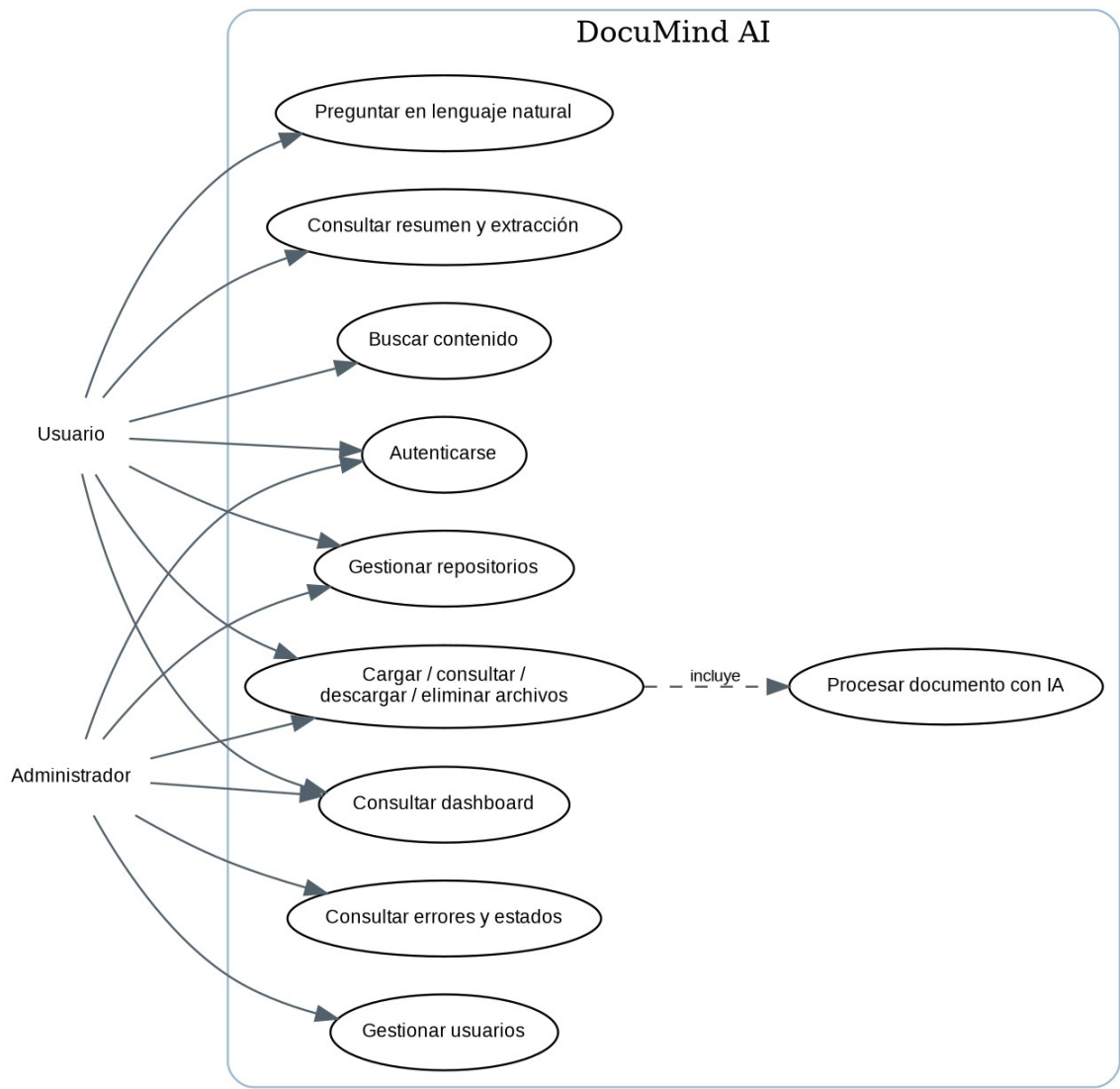


Figura 2. Diagrama de casos de uso de alto nivel.

6. Diagrama de componentes

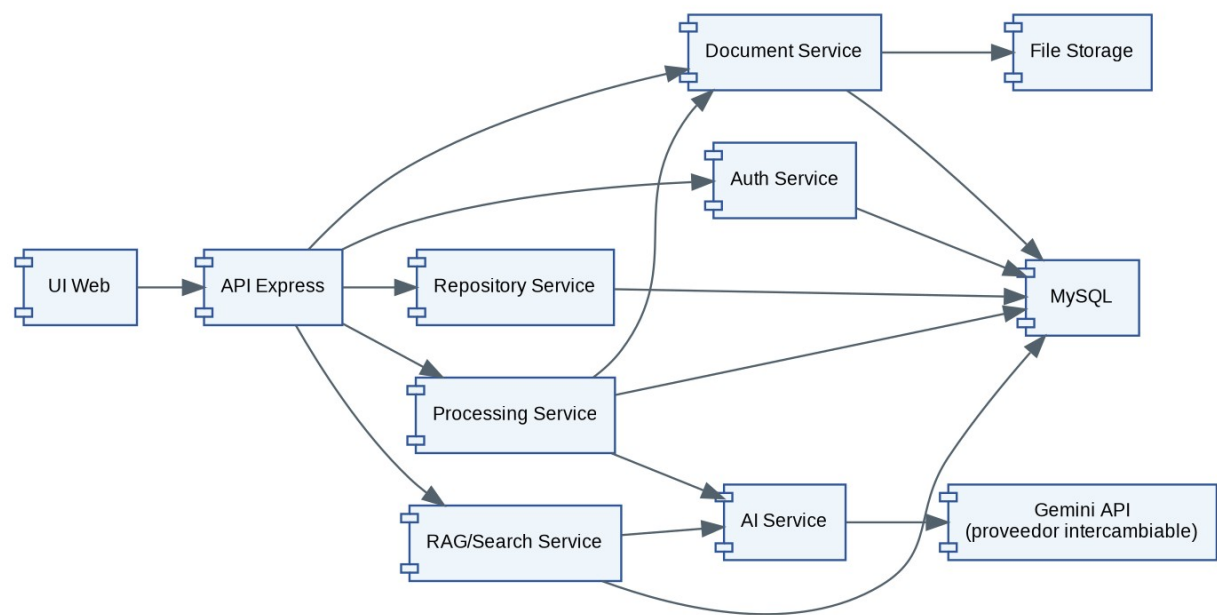


Figura 3. Componentes principales y dependencias.

Componente	Descripción
Auth Service	Registro/login, hash de contraseña, emisión y validación de JWT.
Repository Service	CRUD de repositorios y control de propiedad/acceso.
Document Service	Carga, descarga, consulta, eliminación y metadatos de archivos.
Processing Service	Orquesta extracción, análisis IA, estados y errores.
AI Service	Abstrae clasificación, resumen, extracción y generación de respuestas.
RAG/Search Service	Búsqueda textual, embeddings, similitud y contexto para preguntas.
MySQL	Persistencia de usuarios, documentos, resultados, fragmentos, consultas y errores.
File Storage	Persistencia física del archivo original.

7. Diagrama de despliegue

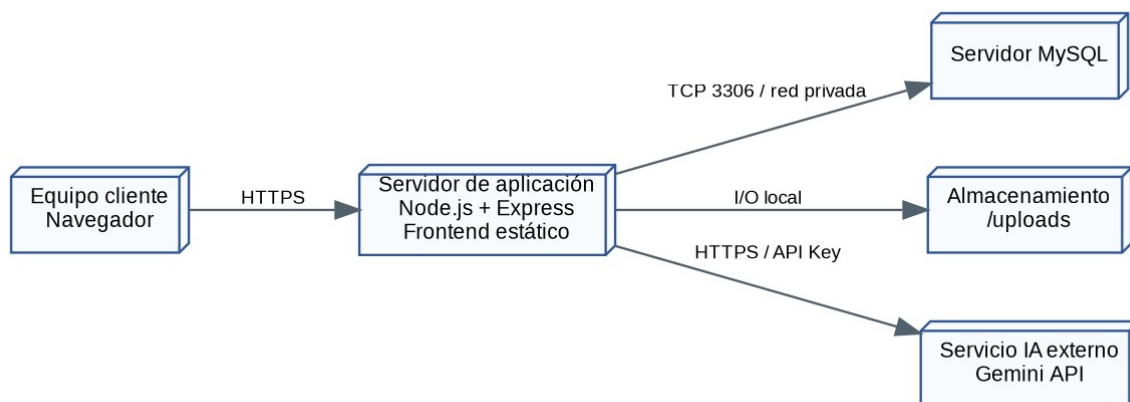


Figura 4. Despliegue propuesto para el prototipo.

En el entorno de desarrollo, aplicación y MySQL pueden ejecutarse en el mismo equipo. Para una implementación futura, se recomienda separar servidor de aplicación, base de datos y almacenamiento de objetos, manteniendo comunicación HTTPS y credenciales por variables de entorno.

8. Diagramas de secuencia

8.1 Carga y procesamiento de un documento

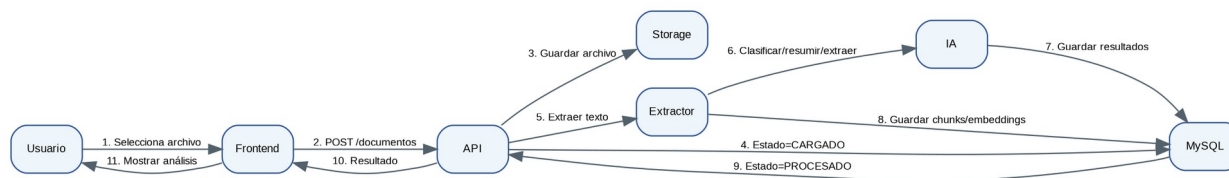


Figura 5. Secuencia de carga y procesamiento.

8.2 Pregunta en lenguaje natural mediante RAG

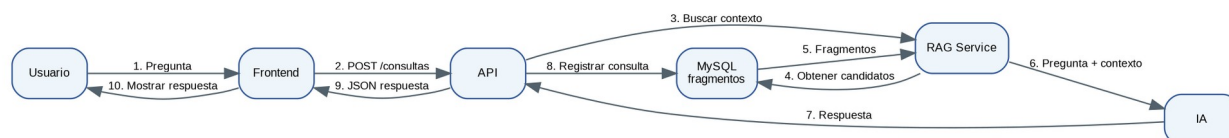


Figura 6. Secuencia de consulta documental con recuperación de contexto.

9. Modelo de datos

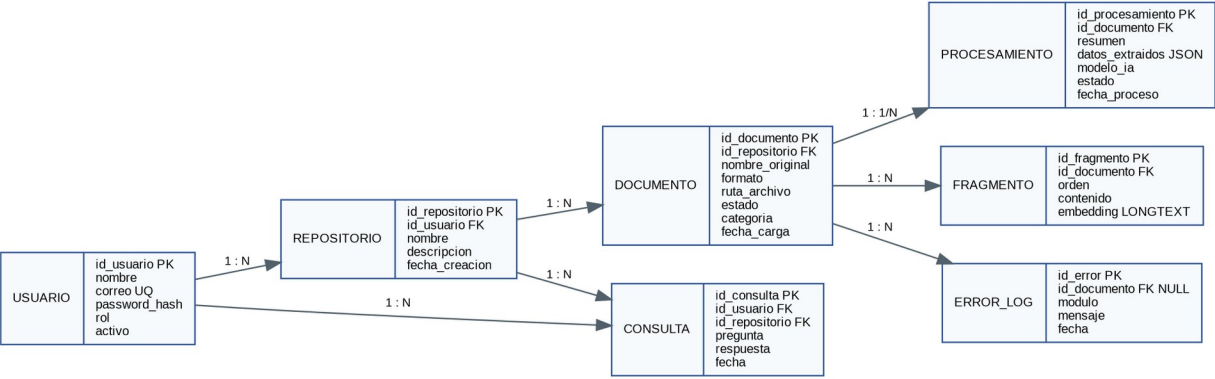


Figura 7. Modelo entidad-relación propuesto.

El modelo separa el archivo físico de sus metadatos y resultados de procesamiento. La entidad FRAGMENTO almacena porciones del contenido y el embedding asociado, permitiendo búsquedas semánticas sobre el repositorio. CONSULTA conserva una bitácora de preguntas y respuestas; ERROR_LOG registra fallos de procesamiento.

10. Diccionario de datos

10.1 Tabla USUARIO

Campo	Tipo	Restricción	Descripción
id_usuario	INT	PK, AI	Identificador.
nombre	VARCHAR(120)	NN	Nombre completo.
correo	VARCHAR(150)	NN, UQ	Correo de acceso.
password_hash	VARCHAR(255)	NN	Hash de contraseña.
rol	VARCHAR(20)	NN	USUARIO o ADMIN.
activo	BOOLEAN	NN	Estado del usuario.
fecha_creacion	DATETIME	NN	Fecha de registro.

10.2 Tabla REPOSITORIO

Campo	Tipo	Restricción	Descripción
id_repositorio	INT	PK, AI	Identificador.
id_usuario	INT	FK, NN	Propietario.
nombre	VARCHAR(120)	NN	Nombre del repositorio.
descripcion	VARCHAR(500)	NULL	Descripción.
fecha_creacion	DATETIME	NN	Fecha de creación.

10.3 Tabla DOCUMENTO

Campo	Tipo	Restricción	Descripción
id_documento	INT	PK, AI	Identificador.
id_repositorio	INT	FK, NN	Repositorio.
nombre_original	VARCHAR(255)	NN	Nombre original.
nombre_almacenado	VARCHAR(255)	NN	Nombre físico único.
formato	VARCHAR(10)	NN	PDF, DOCX o TXT.
mime_type	VARCHAR(100)	NN	Tipo MIME.
tamano_bytes	BIGINT	NN	Tamaño.
ruta_archivo	VARCHAR(500)	NN	Ubicación física.
estado	VARCHAR(30)	NN	Estado de procesamiento.
categoria	VARCHAR(60)	NULL	Factura, Contrato,

			Informe u otra.
fecha_carga	DATETIME	NN	Fecha de carga.

10.4 Tabla PROCESAMIENTO

Campo	Tipo	Restricción	Descripción
id_procesamiento	INT	PK, AI	Identificador.
id_documento	INT	FK, NN	Documento.
texto_extraido	LONGTEXT	NULL	Contenido extraído.
resumen	LONGTEXT	NULL	Resumen IA.
datos_extraidos	JSON	NULL	Campos estructurados.
modelo_ia	VARCHAR(100)	NULL	Modelo/proveedor utilizado.
estado	VARCHAR(30)	NN	Resultado.
fecha_proceso	DATETIME	NULL	Fecha de proceso.

10.5 Tabla FRAGMENTO

Campo	Tipo	Restricción	Descripción
id_fragmento	INT	PK, AI	Identificador.
id_documento	INT	FK, NN	Documento.
orden	INT	NN	Secuencia.
contenido	TEXT	NN	Texto del chunk.
embedding	LONGTEXT	NULL	Vector serializado en JSON.

10.6 Tabla CONSULTA

Campo	Tipo	Restricción	Descripción
id_consulta	INT	PK, AI	Identificador.
id_usuario	INT	FK, NN	Usuario.
id_repositorio	INT	FK, NN	Repositorio consultado.
pregunta	TEXT	NN	Pregunta.
respuesta	LONGTEXT	NN	Respuesta generada.
fecha	DATETIME	NN	Fecha de consulta.

10.7 Tabla ERROR_LOG

Campo	Tipo	Restricción	Descripción
id_error	INT	PK, AI	Identificador.
id_documento	INT	FK, NULL	Documento relacionado.
modulo	VARCHAR(80)	NN	Componente origen.
mensaje	TEXT	NN	Detalle del error.
stack_resumido	TEXT	NULL	Información técnica.
fecha	DATETIME	NN	Fecha.

11. Diseño de API y servicios

Método	Endpoint	Propósito
POST	/api/auth/registro	Crear usuario.
POST	/api/auth/login	Autenticar y devolver JWT.
GET	/api/repositorios	Listar repositorios del usuario.
POST	/api/repositorios	Crear repositorio.
PATCH	/api/repositorios/:id	Editar repositorio.
DELETE	/api/repositorios/:id	Eliminar repositorio según reglas.
POST	/api/repositorios/:id/documentos	Cargar documento y lanzar procesamiento.
GET	/api/repositorios/:id/documentos	Listar documentos.
GET	/api/documentos/:id	Consultar metadatos, resumen y extracción.

GET	/api/documentos/:id/descargar	Descargar original.
DELETE	/api/documentos/:id	Eliminar documento y artefactos asociados.
GET	/api/busqueda?q=...	Buscar texto/metadatos.
POST	/api/consultas	Pregunta en lenguaje natural con contexto RAG.
GET	/api/dashboard	Indicadores del repositorio.
GET	/api/admin/errores	Consultar errores de procesamiento.

12. Diseño de interfaces y prototipos

Los prototipos se plantean con navegación simple y consistente. La versión inicial prioriza funcionalidad y claridad sobre efectos visuales complejos.

Login

+-----+

| DocuMind AI |

| Correo: [] |

| Contraseña: [] |

| [Iniciar sesión] |

+-----+

Dashboard

+-----+

| DocuMind AI | Repositorios | Buscar | Salir |

|-----|

| Documentos: 30 | Procesados: 27 | Error: 1 |

| [Contratos 10] [Facturas 10] [Informes 10] |

| Actividad reciente / estados |

+-----+

Repositorio

+-----+

| Repositorio: Documentos Empresa |

| [Cargar archivo] [Buscar] [Preguntar IA] |

|-----|

| contrato_01.pdf PROCESADO Contrato |

| factura_02.docx PROCESANDO - |

| informe_03.txt ERROR - |

+-----+

Detalle documento

+-----+

| contrato_01.pdf Categoría: Contrato |

|-----|

| Resumen |

| [texto generado...] |

| Datos extraídos |

| Partes | Fecha | Valor | Vigencia |

| [Descargar] [Eliminar] |

+-----+

Consulta IA

+-----+
| Preguntar sobre el repositorio |
| [¿Qué contratos vencen este año?_____] |
| [Preguntar] |
+-----+
| Respuesta basada en documentos recuperados... |
| Fuentes: contrato_01.pdf, contrato_04.pdf |
+-----+

13. Flujo de procesamiento documental

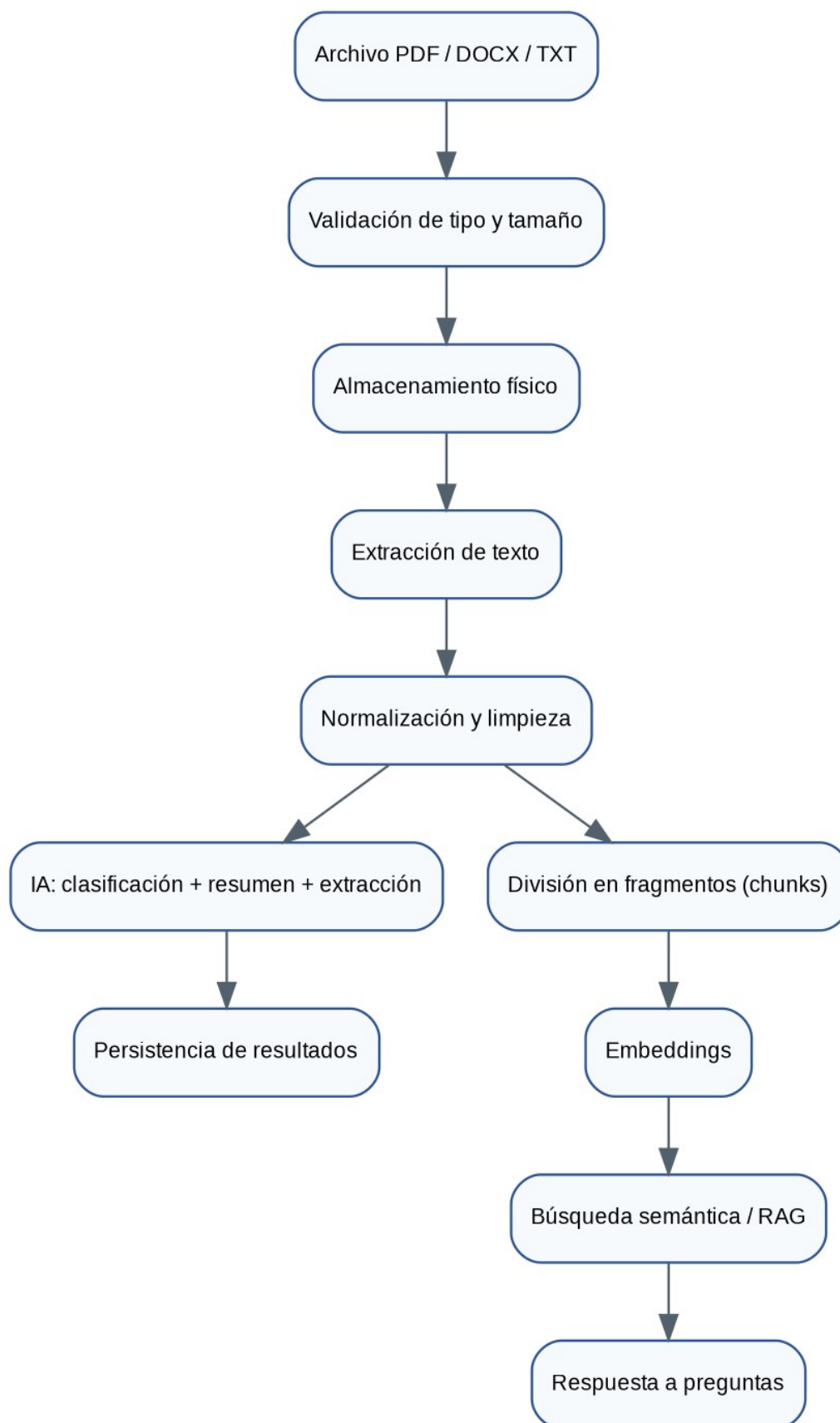


Figura 8. Flujo de procesamiento desde la carga hasta la consulta.

Etapa	Entrada	Salida
Validación	Archivo recibido	Archivo aceptado o error de validación.
Almacenamiento	Archivo válido	Ruta física y metadatos.
Extracción	PDF/DOCX/TXT	Texto plano normalizado.
Clasificación IA	Texto	Categoría y confianza/justificación.
Resumen IA	Texto	Resumen ejecutivo.
Extracción IA	Texto + categoría	JSON con campos relevantes.
Chunking	Texto normalizado	Fragmentos ordenados.
Embeddings	Fragmentos	Vectores para similitud.
RAG	Pregunta + fragmentos	Contexto recuperado y respuesta.

14. Diseño de la integración con IA

Se propone un AI Service como fachada interna. El resto del sistema no invoca directamente al proveedor. Esto permite cambiar entre Gemini, OpenAI u otro servicio sin modificar controladores ni pantallas.

Operación	Prompt/entrada	Salida esperada
Clasificar	Texto del documento + categorías permitidas	JSON: categoría, justificación.
Resumir	Texto del documento	Resumen de longitud controlada.
Extraer	Texto + esquema por categoría	JSON estructurado según tipo documental.
Responder	Pregunta + contexto recuperado	Respuesta basada exclusivamente en el contexto proporcionado.

Para facturas se espera extraer número, proveedor, fecha, valor y concepto; para contratos, partes, fecha, objeto, valor y vigencia; para informes, título, fecha, responsable, tema y conclusiones. El resultado estructurado se valida antes de persistirlo.

15. Diseño de búsqueda inteligente y RAG

Para el corpus académico mínimo de 30 documentos se implementará un RAG simplificado. El texto extraído se divide en fragmentos de tamaño controlado con una pequeña superposición. Cada fragmento recibe un embedding. Los vectores pueden almacenarse serializados en MySQL y la similitud coseno calcularse en Node.js. Esta solución evita introducir una base vectorial adicional para un conjunto pequeño, pero mantiene el concepto de recuperación semántica.

- 1. Normalizar y dividir el texto en chunks.
- 2. Generar embedding por fragmento.
- 3. Guardar contenido, orden y vector.
- 4. Generar embedding de la pregunta.
- 5. Calcular similitud con los fragmentos del repositorio.
- 6. Seleccionar los fragmentos con mayor similitud.
- 7. Construir un prompt con pregunta + contexto.
- 8. Generar respuesta y mostrar documentos fuente.

16. Decisiones tecnológicas y justificación

Tecnología	Decisión	Justificación
HTML/CSS/JavaScript	Frontend sin framework obligatorio	Reduce complejidad y facilita comprensión por todo el equipo.
Node.js + Express	Backend REST	Tecnología conocida, rápida para APIs y manejo de archivos.

MySQL	Base de datos principal	Adecuada para usuarios, repositorios, metadatos, estados y resultados estructurados.
Multer	Carga multipart	Integración directa con Express.
pdf-parse	Extracción PDF	Adecuado para PDFs con texto digital.
mammoth	Extracción DOCX	Convierte contenido DOCX a texto de forma simple.
fs	Lectura TXT	Nativo de Node.js.
JWT + bcrypt	Autenticación	Separación stateless y almacenamiento seguro de contraseñas.
Gemini API mediante AI Service	IA generativa	Proveedor sustituible; clasificación, resumen, extracción y respuestas.
RAG simplificado	Consulta documental	Permite responder usando contexto de documentos sin entrenar un modelo propio.

17. Diseño básico de seguridad

- Contraseñas almacenadas únicamente como hash bcrypt.
- JWT con expiración y middleware de autorización.
- Roles USUARIO y ADMIN para proteger endpoints administrativos.
- API keys y credenciales de MySQL en .env; .env incluido en .gitignore.
- Validación de MIME, extensión y tamaño antes de guardar archivos.
- Renombre interno de archivos para evitar colisiones y nombres maliciosos.
- No permitir rutas provistas directamente por el usuario.
- Consultas SQL parametrizadas.
- Limitar el texto y tamaño enviado a la IA para controlar costos y errores.
- No exponer documentos de otros usuarios; validar propiedad del repositorio antes de operar.
- Registrar errores técnicos sin mostrar stack traces sensibles al usuario final.

18. Gestión de estados y errores

Estado	Significado
CARGADO	Archivo almacenado, pendiente de extracción.
EXTRAYENDO	Extracción de texto en curso.
PROCESANDO_IA	Clasificación/resumen/extracción en curso.
PROCESADO	Proceso finalizado correctamente.
ERROR	Proceso detenido; existe un registro en ERROR_LOG.

Cada transición actualiza el documento para que el dashboard pueda mostrar pendientes, procesados y errores. Si una etapa falla, el archivo no se elimina automáticamente; queda disponible para diagnóstico o reproceso.

19. Consideraciones de escalabilidad

- Para 30 documentos, almacenamiento local y similitud en Node.js son suficientes.
- Para mayor volumen, migrar archivos a almacenamiento de objetos y embeddings a una base vectorial.
- Procesamiento de documentos puede migrarse a una cola de trabajos para no bloquear solicitudes HTTP.
- Los servicios de IA deben incorporar límites de tasa, reintentos y control de costos.
- El diseño modular permite reemplazar componentes sin rediseñar toda la aplicación.

20. Trazabilidad diseño-requisitos

Necesidad funcional	Elemento de diseño que la soporta
---------------------	-----------------------------------

Autenticación y usuarios	Auth Service, JWT, bcrypt, tabla USUARIO.
Repositorios/carpetas	Repository Service, tabla REPOSITORIO, endpoints REST.
Carga/consulta/descarga/eliminación	Document Service, Multer, almacenamiento /uploads.
PDF, DOCX, TXT	Extractor con pdf-parse, mammoth y fs.
Procesamiento mediante IA	Processing Service + AI Service.
Tres categorías	Prompts y esquema Factura/Contrato/Informe.
Resumen por documento	AI Service + PROCESAMIENTO.resumen.
Extracción de datos	AI Service + PROCESAMIENTO.datos_extraidos.
Búsqueda de contenido	Search/RAG Service + texto extraído.
Lenguaje natural	RAG Service + AI Service + CONSULTA.
Dashboard	Endpoints de métricas sobre MySQL.
Errores y estados	DOCUMENTO.estado + ERROR_LOG.

21. Conclusiones

El diseño propuesto cubre las funciones obligatorias del proyecto y mantiene una complejidad adecuada para un prototipo universitario. La separación entre almacenamiento, extracción, IA y RAG permite implementar el flujo real exigido, mantener trazabilidad y demostrar durante la sustentación cómo un archivo pasa de ser contenido no estructurado a información clasificada, resumida, extraída y consultable.